Week 8: HTTP Connections in Flutter

From `http.get()` to testable, production-ready clients

Dr Mohamad AOUDE

Introduction to Multiplatform – Semester VIII
Lebanese University – Faculty of Engineering, Branch II

Learning objectives

By the end of this lecture you will be able to:

- **Understand** the HTTP request/response cycle as seen from a Flutter app.
- **Analyse** `http` and `dio` and choose the right client for a project.
- **Apply** GET and POST requests, JSON parsing and `FutureBuilder` against public sandbox APIs.
- **Evaluate** failures (network, HTTP, parsing) and map each one to a suitable UI response.
- **Create** a layered, injectable network service and test it with controlled testing pairs.

Discuss. When you open an app, what happens in the milliseconds before the UI shows data?

Roadmap

1. The HTTP cycle and why it matters
2. The two clients: `http` and `dio`; setup
3. GET, POST, parsing and `FutureBuilder`
4. Layered architecture and error handling
5. Testing pairs: what and why
6. Unit tests: `MockClient` and Mockito
7. Dio adapter mocks: `http_mock_adapter`
8. In-process mock server with `shelf`
9. Public sandbox APIs
10. Test pyramid, pitfalls, capstone and lab

Guiding question

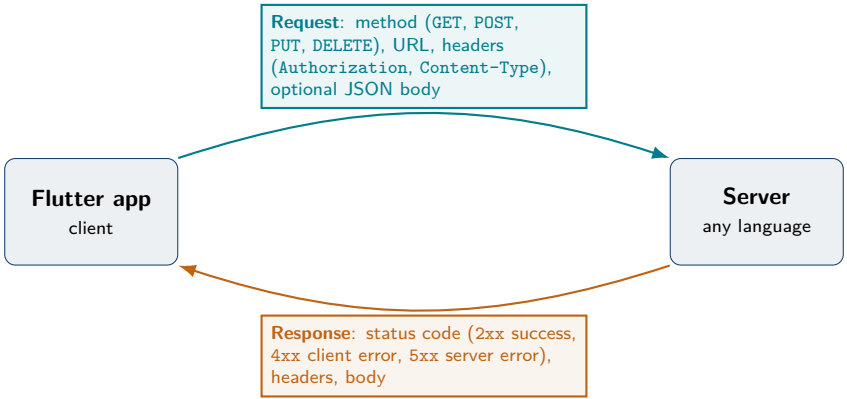
How do you know your HTTP code works *before* a real server exists?

Why HTTP deserves its own lecture

- HTTP is the boundary between your app and every external system: authentication, data sync, media, payments.
- A fragile HTTP layer causes "works on my machine" bugs: flaky tests, silent failures, crashes in production.
- Flutter targets mobile, web and desktop: HTTP code must be portable *and* testable on all of them.
- The core skill is not syntax but **designing for testability**:
  - inject the client,
  - isolate parsing,
  - control the network boundary.

Discuss. What happens to your tests if the API is down?

The HTTP cycle as seen from Dart



HTTP is stateless

Each request carries its required context, such as a token or cookie. Servers can still store application data and session state.

On the wire: one real exchange

Request

```
GET /posts/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json
```

Method + path + host = *what* we ask for.
Headers = metadata. This GET has no body.

Response

```
HTTP/1.1 200 OK
Content-Type: application/json;
    charset=utf-8

{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere ...",
  "body": "quia et suscipit ..."
}
```

Status line + headers + body (a JSON *string*).

Try it now. Open `jsonplaceholder.typicode.com/posts/1` in a browser. Name the four fields, then change 1 to 2, then to 999999.

HTTP methods: what are we asking the server to do?

Method	Purpose	Idempotent?	Classroom action (JSONPlaceholder)
`GET`	Retrieve	yes	Load `/posts` or `/posts/1`
`POST`	Create / submit	no	Create a post: `POST /posts`
`PUT`	Replace the whole resource	yes	Replace post 1: `PUT /posts/1`
`PATCH`	Update some fields	not guaranteed	Change only the title of post 1
`DELETE`	Remove	yes	`DELETE /posts/1`

Idempotent: sending the request twice has the same effect as sending it once. This matters when a request is retried automatically after a timeout.

Discuss. A user changes only their profile picture. Which method fits best, and why?

The `http` package: the minimal baseline

- Published by the Dart team; small, works on every Flutter target.
- Good for simple REST calls, prototypes and learning projects.
- Top-level `http.get()` is hard to replace in tests. **Inject an `http.Client` instead.**
- Close a long-lived client when its owner is disposed.

```dart
import 'package:http/http.dart' as http
    ;

final client = http.Client();
try {
  final response = await client.get(
    Uri.https('jsonplaceholder.typicode
    .com',
        '/albums/1'),
  );
  print(response.statusCode);
} finally {
  client.close(); // release
    connections
}
```

The dio package: when you need more

- Its own client (not built on `package:http`), with `BaseOptions`, **interceptors**, `CancelToken`, `FormData` and progress callbacks.
- Retry and HTTP/2 come from add-on packages, e.g. `dio_smart_retry`, `dio_http2_adapter`.
- Use it for auth pipelines, uploads, cancellation, many endpoints.
- Avoid it for a handful of simple calls: no speculative dependencies.

```dart
import 'package:dio/dio.dart';
import 'package:flutter/foundation.dart
    ';

final dio = Dio(BaseOptions(
  baseUrl: 'https://api.example.com',
  connectTimeout: const Duration(
    seconds: 10),
  receiveTimeout: const Duration(
    seconds: 30),
));

dio.interceptors.add(
  if (kDebugMode) LogInterceptor(
    responseBody: true),
);
```

Discuss. A news reader with one list endpoint: `http` or `dio`? An enterprise app with an auth token on every request and global error logging?

Setup: dependency and platform permissions

```
flutter pub add http      # or: dio
flutter pub add --dev mockito
    build_runner
```

- **Android**: `INTERNET` permission in the *main* manifest (debug builds add it for you; release builds do not).
- **iOS / macOS**: ATS requires HTTPS. macOS also needs the `network.client` entitlement.
- **Web**: the server must allow the browser's origin (CORS).

```
<!-- android/app/src/main/
    AndroidManifest.xml -->
<uses-permission
    android:name="android.permission.
    INTERNET" />
```

Local testing over plain HTTP

Prefer `NSAllowsLocalNetworking` on iOS and a debug-only network security config on Android, **not** "allow arbitrary loads". From the Android emulator, the host machine is `10.0.2.2`, not `localhost`.

Try it now. Add `http` to a scratch project and run `flutter pub get`.

Anatomy of a request: what you control

Part	Meaning	`http` / `dio`
Method	`GET`, `POST`, `PUT`, `PATCH`, `DELETE`: semantics and idempotency	`client.post(...)` / `dio.post(...)`
Headers	`Content-Type`, `Accept`, `Authorization`	`headers:` per call / `BaseOptions.headers`, interceptors
Body	JSON (`jsonEncode`), form fields, multipart; must match `Content-Type`	`body:` string / `data:` map or `FormData`
Timeouts	Connect, send, receive; otherwise a "mysterious hang"	`.timeout(...)` on the future / `BaseOptions`
Status	`2xx` success; `4xx` client error; `5xx` server error	check `statusCode` / `DioException` on bad status

Parsing and modelling: keep the boundary thin

- Never let a raw `Map<String, dynamic>` reach widgets.
- Separate **transport** (HTTP call) from **transformation** (JSON $\rightarrow$ model).
- The function *receives* its client: this one choice enables every test that follows.

```dart
class Album {
  const Album({required this.id, required this.title});
  final int id;
  final String title;

  factory Album.fromJson(Map<String, dynamic> json) =>
      Album(id: json['id'] as int, title: json['title']
     as String);
}

Future<Album> fetchAlbum(http.Client client) async {
  final res = await client.get(
      Uri.https('jsonplaceholder.typicode.com', '/albums
     /1'));
  if (res.statusCode != 200) throw HttpFailure(res.
     statusCode);
  return Album.fromJson(jsonDecode(res.body) as Map<
     String, dynamic>);
}
```

GET a list

- `Uri.https(host, path, query)` builds and encodes the URL; no string concatenation.
- Check the status *before* decoding.
- Check the JSON shape: a list is not a map.
- The client is a parameter, as before.

```dart
Future<List<Post>> fetchPosts(http.Client client) async
    {
  final url = Uri.https('jsonplaceholder.typicode.com',
      '/posts');
  final response = await client.get(url);
     // 1. request
  if (response.statusCode != 200) {
     // 2. status
    throw HttpFailure(response.statusCode);
  }
  final data = jsonDecode(response.body);
     // 3. parse
  if (data is! List) {
    throw const FormatException('Expected a JSON list')
     ;
  }
  return data
      .map((e) => Post.fromJson(e as Map<String,
     dynamic>))
      .toList();
}
```

Sending data: a POST request

- Set `Content-Type`: the server needs to know the body is JSON.
- `jsonEncode` the map: a Dart `Map` is not a JSON string.
- "Created" is 201, not 200.
- JSONPlaceholder answers with `id: 101`, but nothing is stored.

```dart
Future<Post> createPost(
    http.Client client, String title, String body)
    async {
  final response = await client.post(
    Uri.https('jsonplaceholder.typicode.com', '/posts')
    ,
    headers: {'Content-Type': 'application/json;
    charset=UTF-8'},
    body: jsonEncode({'title': title, 'body': body, '
    userId': 1}),
  );
  if (response.statusCode != 201) {
    throw HttpFailure(response.statusCode);
  }
  return Post.fromJson(
      jsonDecode(response.body) as Map<String, dynamic
    >);
}
```

Discuss. Create a post, then `GET /posts/101`. What comes back, and what does a 201 really prove?

PUT, PATCH and DELETE

- Same pattern: URI, headers, body, status check.
- `PUT` sends the *whole* resource; `PATCH` sends only the changed fields.
- Success is often 200 or 204 `No Content` (empty body).
- The real server decides persistence and the response format: read its docs.

```
final uri = Uri.https('jsonplaceholder.typicode
    .com', '/posts/1');
const json = {'Content-Type': 'application/json
    ; charset=UTF-8'};

await client.put(uri, headers: json, body:
    jsonEncode(
    {'id': 1, 'userId': 1, 'title': 'New', '
    body': 'Whole post'}));

await client.patch(uri, headers: json,
    body: jsonEncode({'title': 'Only the title'
    }));

final res = await client.delete(uri);
if (res.statusCode != 200 && res.statusCode !=
    204) {
  throw HttpFailure(res.statusCode);
}
```

Connecting UI and network: `FutureBuilder`

- Requests are asynchronous; the UI must keep rendering.
- Design **four states**: loading → progress indicator; error → message + Retry; empty → "nothing found"; success → list.
- **Create the future once** (`initState` or a retry handler), never inside `build`: every rebuild would send a new request.

```
late Future<List<Post>> _posts;

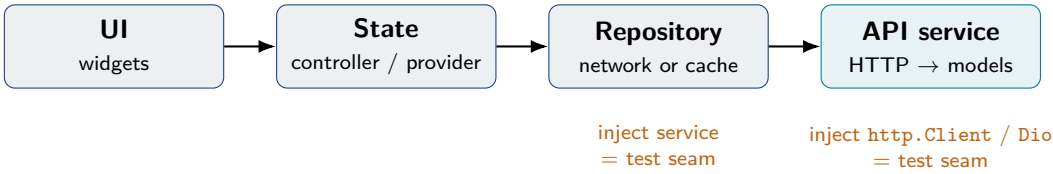
@override
void initState() {
  super.initState();
  _posts = widget.service.fetchPosts();
}

void _retry() =>
    setState(() => _posts = widget.service.fetchPosts
     ());
```

FutureBuilder: render the four states

```
@override
Widget build(BuildContext context) => FutureBuilder<List<Post>>(
  future: _posts,
  builder: (context, snapshot) {
    if (snapshot.connectionState != ConnectionState.done) {
      return const Center(child: CircularProgressIndicator());
    }
    if (snapshot.hasError) return RetryView(onRetry: _retry);
    final posts = snapshot.data!;
    if (posts.isEmpty) return const Text('No posts yet');
    return ListView.builder(
      itemCount: posts.length,
      itemBuilder: (_, i) => ListTile(title: Text(posts[i].title)),
    );
  },
);
```

Architecture: separation of concerns



- **API service**: raw requests; returns Dart objects or typed failures.
- **Repository**: decides between network and local cache.
- **UI**: listens to state and renders. **No `http.get` inside `build()`.**

Discuss. If a widget calls `http.get` directly, how would you test its error state?

Error handling: model failure, don't guess

- **Transport**: timeout, DNS, connection refused.
- **HTTP**: 401, 404, 422, 500 ...
- **Parsing**: malformed JSON or missing fields.
- `http`: check `statusCode`; catch `FormatException`, `TypeError`, `TimeoutException`.
- `dio`: branch on `DioException.type`.
- Never swallow errors: throw a domain exception or return a typed failure.

```
try {
  final res = await dio.get('/albums/1');
  return Album.fromJson(res.data);
} on DioException catch (e) {
  switch (e.type) {
    case DioExceptionType.connectionTimeout:
    case DioExceptionType.receiveTimeout:
      throw const NetworkTimeout();
    case DioExceptionType.badResponse:
      throw HttpFailure(e.response!.
     statusCode!);
    case DioExceptionType.cancel:
      throw const RequestCancelled();
    default:
      throw const NetworkUnavailable();
  }
}
```

Discuss. What should the UI do for a 401, a 500 and a timeout? Are the answers the same?

From failure type to user experience

Failure	Dart signal	What the user should see
No connection / DNS	`ClientException`, `SocketException`*	Offline illustration and a **Retry** button
Timeout	`TimeoutException`	"Taking too long" and Retry
400 / 422	status code + error body	The server's validation message next to the field
401	status code	Session expired: go back to login
403 / 404	status code	"Not allowed" / "Not found", no Retry
429	status code	"Too many requests": wait (honour `Retry-After`)
500 / 503	status code	"Our servers are busy, try again later" (retry with back-off)
Bad JSON	`FormatException`, `TypeError`	Generic error to the user; log it as a bug

*`SocketException` comes from `dart:io` and does not exist on the web; `package:http` reports transport errors as `ClientException`.

Lab: break your application deliberately

Point the post viewer at each URL and record what happens:

1. `https://no-such-host.invalid/`
2. `https://httpbin.org/status/404`
3. `https://httpbin.org/status/503`
4. `https://httpbin.org/delay/3` with a 2 s timeout
5. Any URL with Wi-Fi / mobile data switched off

```
try {
  final res = await client
      .get(Uri.parse('https://httpbin.org/
     delay/3'))
      .timeout(const Duration(seconds: 2));
  if (res.statusCode != 200) {
    throw HttpFailure(res.statusCode);
  }
} on TimeoutException {
  showMessage('The server is taking too long
     .');
} on http.ClientException {
  showMessage('No connection. Check your
     network.');
} on HttpFailure catch (e) {
  showMessage(messageFor(e.statusCode));
}
```

Discuss. Which failures return a status code, and which prevent any HTTP response from arriving?

The testing-pair concept

A **testing pair** = the code under test + a *controlled* network boundary.

Pair	Fidelity	Speed	Best for
Real API (sandbox)	Highest	Slowest	Smoke tests, learning
In-process server (`shelf`)	High	Fast	Real sockets, headers, serialisation
Adapter mock (`http_mock_adapter`)	Medium	Very fast	Dio pipelines and interceptors
`MockClient` / Mockito	Lower	Fastest	Pure unit tests, edge cases

Key idea

No single pair is "best". Production test suites **combine** them.

Pair 1a: `MockClient` from `package:http/testing.dart`

- Ships with `http`: no code generation.
- A function maps each request to a response.
- Inspect the request: method, URL, headers.
- Week 7's `PostService` tests use this approach.

```dart
import 'package:http/http.dart' as http;
import 'package:http/testing.dart';
import 'package:test/test.dart';

void main() {
  test('returns Album on 200', () async {
    final client = MockClient((request) async {
      expect(request.url.path, '/albums/1');
      return http.Response(
          '{"id": 1, "title": "Test Album"}', 200);
    });
    final album = await fetchAlbum(client);
    expect(album.title, 'Test Album');
  });

  test('throws on 404', () async {
    final client =
        MockClient((_) async => http.Response('', 404))
    ;
    await expectLater(fetchAlbum(client), throwsA(isA<
     HttpFailure>()));
  });
}
```

Pair 1b: Mockito on `http.Client`

- Null-safe Mockito **generates** the mock: `@GenerateMocks` + `build_runner`.
- Stub with `when`, verify calls with `verify`.
- Cover 200, 404, 500 and malformed JSON in milliseconds.

```
flutter pub add --dev mockito
     build_runner
dart run build_runner build
```

```dart
import 'package:mockito/annotations.dart';
import 'package:mockito/mockito.dart';
import 'fetch_album_test.mocks.dart';

@GenerateMocks([http.Client])
void main() {
  test('throws FormatException on bad JSON',
    () async {
    final client = MockClient();
    when(client.get(any)).thenAnswer(
        (_) async => http.Response('not json',
     200));
    await expectLater(fetchAlbum(client),
        throwsA(isA<FormatException>()));
    verify(client.get(any)).called(1);
  });
}
```

Pair 2: adapter-level mocking with `http_mock_adapter`

- Replaces Dio's `httpClientAdapter`: no socket is opened.
- Declare request → response pairs: `onGet`, `onPost`, `onPatch`, `onDelete`.
- Dio's real pipeline still runs, so **interceptors are tested too**.

```dart
import 'package:dio/dio.dart';
import 'package:http_mock_adapter/http_mock_adapter.
    dart';
import 'package:test/test.dart';

void main() {
  final dio = Dio(BaseOptions(
      baseUrl: 'https://example.com'));
  final dioAdapter = DioAdapter(dio: dio);

  test('mock GET returns payload', () async {
    dioAdapter.onGet('/status',
        (server) => server.reply(200, {'message': 'ok
    '}));
    final res = await dio.get('/status');
    expect(res.data['message'], 'ok');
  });
}
```

Pair 3: in-process mock server with `shelf`

- A real HTTP server on `localhost`, started by the test; port 0 picks a free port.
- Exercises real sockets, headers, encoding and timeouts; still fast and offline.
- Works for `http` and `dio`: only the base URL changes.
- Runs in VM tests (`test()`); in `testWidgets`, inject a `MockClient` instead.

```dart
import 'dart:io';
import 'package:shelf/shelf.dart';
import 'package:shelf/shelf_io.dart' as io;

late HttpServer server;

setUp(() async {
  server = await io.serve((Request req) {
    if (req.url.path == 'albums/1') {
      return Response.ok('{"id":1,"title":"Local"}',
          headers: {'content-type': 'application/json
      '});
    }
    return Response.notFound('');
  }, InternetAddress.loopbackIPv4, 0);
});
tearDown(() => server.close(force: true));

test('talks to a real socket', () async {
  final client = http.Client();
  addTearDown(client.close);
  final album = await AlbumApi(client,
      'http://127.0.0.1:${server.port}').fetch(1);
  expect(album.title, 'Local');
});
```

Pair 4: public sandbox APIs (1/2) — CRUD, protocol, catalogue

Service	Try	Best for
JSONPlaceholder	`/posts`, `/users`, `/albums`	No auth, predictable JSON; all five methods (writes are simulated)
httpbin / httpbun	`/get`, `/headers`, `/status/404`, `/delay/3`	Seeing what the server *received*; forcing status codes and delays
DummyJSON	`/products`, `/products/search` with `q=phone`, `?limit=10&skip=10`	Realistic catalogue: images, prices, search, pagination

Caveats

Writes on fake CRUD APIs **do not persist**. Public services can be slow, rate-limited or down: check them before class, never send real credentials, and keep them out of your fast test suite.

Pair 4: public sandbox APIs (2/2) — auth, nesting, keys

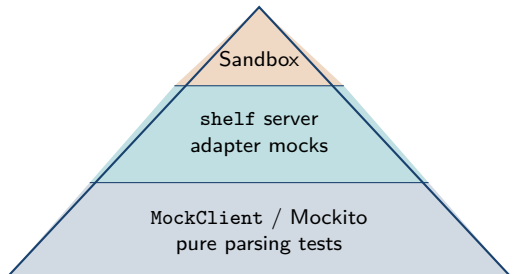
- **ReqRes** — paginated users (`/api/users?page=2`) and `/api/login`: a missing password returns 400 with an error message. Anonymous demos are rate-limited; account APIs use keys. Check the current docs.
- **PokeAPI** — deeply nested JSON (types, abilities, stats): practise nested `fromJson`.
- **OpenWeatherMap** — needs a free API key (`appid`): learn 401 for a bad key and 429 for rate limits. A key shipped in an app is extractable; production apps call through a backend.

```
// pokeapi.co/api/v2/pokemon/pikachu (
    excerpt)
{
  "name": "pikachu",
  "types": [
    {"slot": 1,
     "type": {"name": "electric", "url
     ": "..."}}
  ],
  "stats": [
    {"base_stat": 35,
     "stat": {"name": "hp", "url":
     "..."}}
  ]
}
```

Try it now. Write `Pokemon`, `PokemonType` and `Stat` classes with `fromJson` from this excerpt alone, before reading the docs.

Discuss. Moving from a plain list (JSONPlaceholder) to a paginated one (ReqRes): what state do you need? (page, items so far, "has more", loading flag)

A practical test pyramid for HTTP



- **Base:** unit tests; run on every save.
- **Middle:** Dio pipelines, local server, widget tests with injected clients; run in CI on every push.
- **Top:** live sandbox smoke tests; nightly or before release.

Goal

Confidence per second, not 100% coverage at one level.

Best practices and common pitfalls

Do

- Inject an `http.Client` or `Dio`; avoid top-level calls in production code.
- Close clients when their owner is disposed.
- Set timeouts on every production client.
- Parse JSON in dedicated functions, not in widgets.
- Keep base URLs (dev/staging/prod) in configuration.

Avoid

- Calling a test against a live API a "unit test".
- Mocking `Dio` itself and wondering why interceptors never run.
- Assuming fake CRUD APIs persist writes.
- Catching `DioException` without branching on `type`.
- Hand-written Mockito mocks under null safety.

Discuss. Spot the bug: refactor a widget that calls `http.get()` inside `build` so that it can be tested.

Capstone exercise: build and test every pair

1. Implement `fetchAlbums()` with an injected `http.Client` against JSONPlaceholder `/albums`.
2. Test with `MockClient` or Mockito: 200 with two albums, 404, 500, malformed JSON.
3. Implement the same endpoint with `dio`; add a retry interceptor; test it with `http_mock_adapter`.
4. Write a `testWidgets` that pumps a `ListView` fed by an injected fake client and asserts the items appear.
5. Run the service against a local `shelf` server.
6. Bonus: a fake client that responds after 5 s and a 1 s timeout; then try `httpbin.org/delay/5` by hand.

Deliverable

A short README stating which pair you used for each test, and why.

Lab: Product Explorer (pairs)

Build on DummyJSON (`https://dummyjson.com/products`):

1. Fetch products into typed `Product` objects in a `ProductService` that receives an `http.Client`.
2. Show name, price and thumbnail in a `ListView.builder`.
3. Handle loading, empty, error and success; add Retry.
4. Search with `/products/search?q=...`
5. "Load more" with `limit` and `skip`.
6. Test the service with `MockClient`: 200, empty list, 500, bad JSON.

Extension: a login screen on ReqRes `/api/login`; trigger the 400 once on purpose and show its message.

Criterion	Weight
Requests and status validation	25%
JSON parsing and models	20%
UI states and state handling	25%
Error handling and retry	20%
Organisation and readability	10%

Summary and key takeaways

- HTTP code is not testable by accident: **inject clients, isolate parsing, model errors**.
- Start with `http`; reach for `dio` when interceptors, cancellation or uploads justify it.
- Use testing pairs:
  - `MockClient` / Mockito for speed and edge cases,
  - `http_mock_adapter` for Dio pipelines,
  - a local `shelf` server for real-socket behaviour,
  - public sandboxes for realism.
- A good suite is **layered**: combine pairs to balance fidelity and feedback speed.
- **Next steps**: auth tokens via Dio interceptors, `retrofit` for type-safe clients, a local cache (e.g. `sqflite`) for offline use.

Discuss. What would change if this were a banking app rather than a product catalogue? (authentication, HTTPS only, privacy, stricter reliability)

Exit quiz

1. Which HTTP method normally retrieves data? (a) POST (b) GET (c) DELETE
2. What does `await` do? (a) blocks the UI thread (b) waits asynchronously for a `Future` (c) retries
3. A `404` means: (a) server unreachable (b) resource not found (c) invalid JSON
4. Why not start a request inside `build()`? (a) it repeats on every rebuild (b) HTTP is forbidden in widgets
5. A simulated POST returns `201`. This proves: (a) the data is saved permanently (b) the server acknowledged the creation
6. Why does `fetchPosts` take an `http.Client` parameter? (a) speed (b) so tests can pass a controlled client

References

- Flutter cookbook: Fetch data from the internet; Mock dependencies using Mockito.
- Packages: http, dio, mockito, http_mock_adapter, shelf.
- Flutter: Networking; Send data to the internet.
- Sandboxes: JSONPlaceholder, httpbin, httpbun, DummyJSON, ReqRes, PokeAPI, OpenWeatherMap.
- Course code: `code/week07/examples/lib/services/post_service.dart` and its tests.